

Introduction to Digital Logic Design Lab

EECS 31L

Lab 2: Arithmetic Logic Unit

Ali Mortada
19849621

Februrary 8th, 2025

1 Objective

The purpose of this lab was to implement and test a basic 32-bit Arithmetic Logic Unit (ALU) in Verilog. The functions of the ALU include bitwise AND, OR, and NOR, signed addition, subtraction, and set less than, and equal comparison.

Below in Fig. 1 is a block diagram for the ALU. It takes in three inputs: two 32-bit inputs A and B to perform the operations on, and a 4-bit select input that chooses which operation to perform. The four outputs include Zero, Overflow, and Carry Out, which are all 1-bit, and Result, which is 32-bit. Result displays the result of the operation, Zero is high when Result is 0 and low otherwise, Overflow is high when there is an overflow in the addition operation and low otherwise, and Carry Out is high when there is a carry out bit in addition and low otherwise.

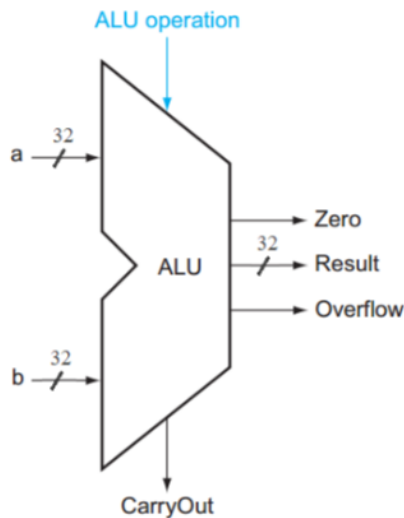


Figure 1: Block diagram for 32-bit ALU.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	Set less than
1100	NOR
1111	Equal comparison

Figure 2: Values of ALU control line and their corresponding operations.

Fig. 2 shows the values of the ALU control line (in this case, the 4-bit select input) and their corresponding operations. Below are the expected outputs for each operation:

- **AND:** Performs a bitwise AND operation on the inputs A and B, outputting the result to the Result bus.
- **OR:** Performs a bitwise OR operation on the inputs A and B, outputting the result to the Result bus.
- **Add:** Performs signed addition on the inputs A and B, outputting the result to the Result bus. If there is overflow, the Overflow bit is set to 1, otherwise it is set to 0. For all other ALU operations, the Overflow bit is set to 0. If the addition results in a carry out bit, then the Carry Out bit is set to 1, otherwise it is set to 0. For all other operations, the Carry Out bit is set to 0.
- **Subtract:** Performs signed subtraction on the inputs A and B, outputting the result to the Result bus.
- **Set Less Than:** Tests the inputs A and B. If A is less than B, then Result is set to 1. Otherwise, Result is set to 0. A and B are both signed.
- **NOR:** Performs a bitwise NOR operation on the inputs A and B, outputting the result to the Result bus.
- **Equal Comparison:** Tests the inputs A and B. If they are equal, then Result is set to 1, otherwise it is set to 0.

2 Procedure

Since the ALU features a select input that performs a different operation on the inputs based on its value, the most straightforward implementation is using a `case` statement. Since sequential logic is used, the outputs Result, Overflow, and Carry Out are declared as type `reg`. The sensitivity list of the `case` statement includes all of the inputs A, B, and Select. For each case other than addition, Carry Out is set to zero to avoid undefined values and to avoid having the value carry over from previous cases. The same applies for Overflow with every case other than addition and subtraction.

2.1 Bitwise Operations and Equal Comparison

The implementations for the bitwise operations AND, OR, and NOR is straightforward. Respectively, the statements used to perform the operations within each case are:

```
ALU_Out = A_in & B_in;    // bitwise AND
ALU_Out = A_in | B_in;    // bitwise OR
ALU_Out = ~(A_in | B_in); // bitwise NOR
```

Similarly, the implementation for the equal comparison is also straightforward. Below is the statement used in its case:

```
ALU_Out = A_in == B_in;
```

2.2 Signed Set Less Than

The implementation for the set less than operation is similar to the equal comparison, except that it is a signed operation. This is done by type casting the inputs A and B to signed integers using the `$signed()` keyword. Below is the statement used in its case:

```
ALU_Out = $signed(A_in) < $signed(B_in);
```

2.3 Signed Addition and Subtraction

The implementation for signed addition is slightly more complicated than that of the bitwise operations and comparator operators. The inputs cannot be cast using the `$signed()` keyword since the Carry Out bit would be incorrectly calculated due to the way Verilog performs signed addition. It also makes detecting overflow difficult. Thus, signed addition in the module is implemented by simulating the way Verilog performs signed addition but manually. This is done by extending the most significant bit of each of the inputs, adding them, assigning the result to the Result bus, and assigning the extra bit to an internal wire. If the value of the extra bit concatenated with the most significant bit of the result is equal to 01 or 10, then there is overflow. The carry out bit is found by performing regular unsigned addition between the inputs.

To do this, some constants and internal wires must be declared outside the `always` loop:

```
// constant declarations
localparam WIDTH = 32;
localparam MSB = WIDTH - 1;

// I/O port definitions
.
.
.

// internal wires
reg extra_bit; // for signed addition
reg [MSB:0] temp_result; // for finding the carry out of signed addition
```

The implementation of signed addition inside the case statement is as follows:

```
begin // signed addition
    // perform addition with "extended bit"
    {extra_bit, ALU_Out} = {A_in[MSB], A_in} + {B_in[MSB], B_in};
    Overflow = ({extra_bit, ALU_Out[MSB]} == 2'b01) | // case for overflow
               ({extra_bit, ALU_Out[MSB]} == 2'b10); // case for underflow
    {Carry_Out, temp_result} = A_in + B_in; // to find carry out bit
end
```

Likewise, the implementation of signed subtraction is as follows:

```
begin // signed subtraction
    // perform subtraction with "extended bit"
    {extra_bit, ALU_Out} = {A_in[MSB], A_in} - {B_in[MSB], B_in};
    Overflow = ({extra_bit, ALU_Out[MSB]} == 2'b01) | // case for overflow
               ({extra_bit, ALU_Out[MSB]} == 2'b10); // case for underflow
    Carry_Out = 1'b0;
end
```

The only difference is that the operation done is subtraction instead of addition and that the Carry Out bit is set to zero.

3 Simulation Results

Eight test cases were provided for the simulation. The first six cases test each of the operations once, while the last two test the equal comparison operation (one case where the inputs are equal and one case where they are not). Listed below are each of the test cases with their expected outputs:

1. A = "32'h086a0c31", B = "32'hd785f148", ALU_Sel = "0000", Expected ALU_Out = "32'h00000000"
2. A = "32'h086a0c31", B = "32'10073fd4", ALU_Sel = "0001", Expected ALU_Out = "32'h186f3ff5"
3. A = "32'ha86a0c31", B = "32'h90073fd4", ALU_Sel = "0010", Expected ALU_Out = "32'h38714c05"
4. A = "32'ha86a0c31", B = "32'h90073fd4", ALU_Sel = "0110", Expected ALU_Out = "32'h1862cc5d"
5. A = "32'ha86a0c31", B = "32'h90073fd4", ALU_Sel = "0111", Expected ALU_Out = "32'h00000000"
6. A = "32'ha86a0c31", B = "32'h90073fd4", ALU_Sel = "1100", Expected ALU_Out = "32'h4790c00a"
7. A = "32'ha86a0c31", B = "32'ha86a0c31", ALU_Sel = "1111", Expected ALU_Out = "32'00000001"
8. A = "32'ha86a0c31", B = "32'h10073fd4", ALU_Sel = "1111", Expected ALU_Out = "32'00000000"

Below, in Fig. 3, is a screenshot of the simulation results.

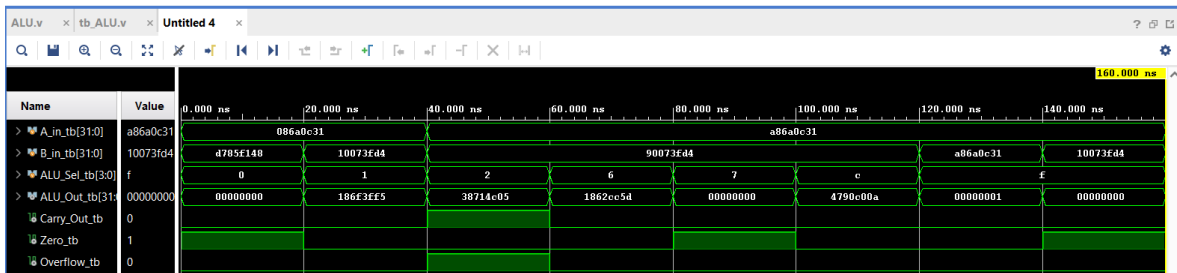


Figure 3: Simulation results for all test cases of the 32-bit ALU.

From the results, we can see that ALU_Out has the expected value for each test case. It is also clear that the Zero output is high whenever ALU_Out is all zero (cases 1, 5, and 8). For the third test case (signed addition), the Carry Out and Overflow signals are both high – this is expected, since adding these two 32-bit positive integers results in a 33-bit integer (meaning there is a carry out) with a leading 1 (meaning that the result is negative). Both bits are also active low for every other case, which fulfills the requirements.